

SIMULADO DE AVALIAÇÃO DE ENGENHARIA DE SOFTWARE

EXERCÍCIOS AVANÇADOS DE GIT E GITHUB

Do histórico local à publicação com GitHub Pages e Cloudflare

Propósito: *resolver problemas reais de rastreabilidade, colaboração, integração e entrega em ritmo de avaliação. Este caderno não contém gabarito nem sequências completas de comandos.*

Orientações de entrega

1. Trabalhe em um repositório criado especificamente para cada exercício.
2. Antes de cada operação que altere o histórico ou publique conteúdo, registre o estado observado com comandos de inspeção adequados.
3. Nunca inclua senha, Personal Access Token, arquivo `.env` real ou outro segredo nas evidências. Credenciais expostas devem ser revogadas, não apenas apagadas do último *commit*.
4. Entregue os artefatos pedidos em cada exercício. Capturas de tela devem mostrar o contexto suficiente para identificar repositório, ramificação e resultado.
5. Justifique decisões de histórico, revisão, aceitação ou não de *pull requests*.
6. **Não vale qualquer ponto na nota.** Mas você deve fazer todas as operações e recolher todas as evidências como se valesse.

EXERCÍCIO 1 · COLABORAÇÃO DISTRIBUÍDA E REVISÃO

Equipe	
Turma	
Data	
Repositório	

Do fork ao pull request

MODALIDADE	TEMPO	FOCO	ENTREGA
Dupla	30-40 min	fork + branch + PR	PR integrado + evidências

Desafio: *produzir uma contribuição revisável entre dois repositórios, responder a uma solicitação de mudanças no mesmo branch e fechar o ciclo sem contaminar a main do fork.*

Cenário

Crie um repositório `seu_usuario/site-disciplina`. A `main` oficial está protegida e só aceita integração por *pull request* aprovado. Uma pessoa será o autor da contribuição; a outra atuará como revisora e mantenedora (administrador do projeto). Este repositório deve conter, no mínimo:

```
site-disciplina/  
├── index.html  
├── css/estilo.css  
├── README.md  
└── participantes.md
```

Tarefas

1. O autor cria um *fork*, clona sua própria cópia e configura o repositório oficial como `upstream`. A dupla deve demonstrar, por inspeção, para onde um *push* será enviado e de onde virão as atualizações oficiais.
2. Partindo da `main` oficial mais recente, O autor cria um branch `feature/<RA>-pagina-contato`. O *branch* deve nascer somente depois de a base local avançar em linha reta e de o *fork* receber a mesma posição.
3. A contribuição deve conter dois *commits* atômicos: o primeiro aluno cria `contato.html` com estrutura HTML acessível; o segundo adiciona o link correspondente à navegação de `index.html`. Nenhum outro arquivo pode entrar na preparação.

4. O autor publica o *branch* no *fork* e abre um *pull request* em modo rascunho. Antes de marcá-lo como pronto, registra os quatro identificadores da comparação: repositório-base, branch-base, repositório-head e branch-head.
5. A descrição do *pull request* deve explicar o que mudou, por que a mudança existe e como foi verificada. Inclua uma captura da página e uma lista de testes reproduzíveis. A aba de arquivos alterados deve comprovar o escopo de dois arquivos.
6. A revisora solicita pelo menos uma correção objetiva de acessibilidade ou navegação. O autor corrige a mesma ramificação, cria um *commit* e publica a atualização sem abrir outro *pull request*. A dupla comprova que o número e a URL da proposta não mudaram.
7. Depois da aprovação, O administrador integra com *squash and merge*. O autor sincroniza a *main* local e a do *fork*, remove as ramificações concluídas e compara o grafo anterior com o posterior à integração.

Critérios de revisão

- O *diff* contém somente contato.html e a alteração necessária em index.html.
- Os *commits* intermediários têm intenção identificável e não contêm arquivos gerados ou credenciais.
- O destino é a *main* do repositório oficial; a origem é o *branch* do *fork* da autora.
- Todas as conversas foram respondidas e a solicitação de mudança foi resolvida antes da integração.
- A versão final pode ser aberta localmente e a navegação não contém links quebrados.

Evidências obrigatórias

- Saída de inspeção dos remotos e grafo que diferencie *main*, *origin/main*, *upstream/main* e o *branch* da tarefa.
- URL do *pull request*, captura da direção *base/head* e histórico da conversa de revisão.
- Capturas do estado antes e depois do novo *push* que atualizou a mesma proposta.
- Grafo final e explicação do efeito de *squash and merge* sobre os commits originais do *branch*.